Java Intro

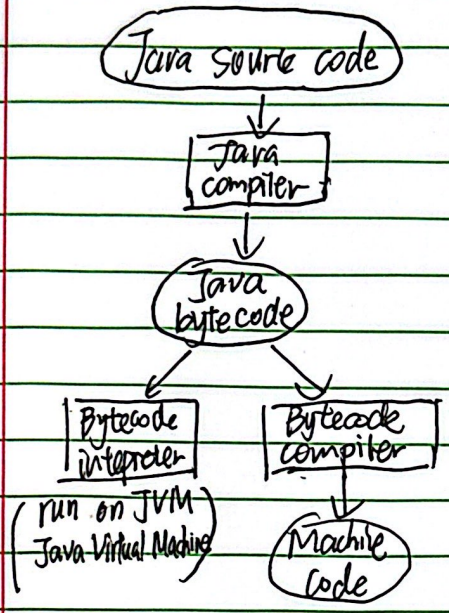
Program Development

Programming lang. level

- Machine lang. — each CPU type has own machine lang.
- Assembly lang.
- High-level lang.
- Fourth-generation lang.

Compiler: software, source code ⟷ target language

Java compiler not tied to particular machine ⇒ architecture-neutral



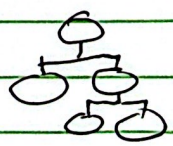
Object Oriented Programming

Object — fundamental entity in Java, represent real-world entities, defined by class.
- state
- behavior

Class — blueprint of object, use methods to define behavior of the object.
main method → entire prog.
multiple objects can be created from same class

Class VS Object
Object → instantiation (实例化) of a class, assigned real space to store value.
Class → occupy no space when defined
Multiple object can derive from same class

One class used to derive another → inheritance
Classes can be organized into hierarchies



Java Programming Language.

case sensitive, use semicolon to end statement, declare var before use.

class → method → statements

Comments: ① // a line ② /* big block of text */ ③ /** javadoc comment */

Identifiers: "words", 可用：字母、数、下划线，$；不能以数开头

class names: Lincoln

constant names: MAXIMUM

Data & Expressions

String

double quotes!!! "xxx"

every String is object → String

- Concatenation

String + String → String

String + num → String

num + String → String

num + num → num

- escape sequence

begin w backslash.

\b backspace

\t tab

\n newline

\r carriage return

\" double quote

\' single quote

\\ backslash

variable

data type + variable name

e.g. int count, temp, result;

final int MIN = 69; // final declares constant

constants

give meaning to unclear literal values

facilitate program maintenance (if used mult. times, it's set in one place)

avoid inadvertent errors

primitive data types

byte, short, int, long — represent integers

float, double — floating pt. num.

char - characters

boolean - boolean.

Type	Size	Min	Max.
byte	8 bits	-128	127
short	16 bits	-32768	32767
int	32 bits	-2147483648	2147483647
long	64 bits	$<-9\times10^{18}$	$>9\times10^{18}$
float	32 bits	$\pm 3.4\times10^{38}$	~ 7 sf
double	64 bits	$\pm 1.7\times10^{308}$	~ 15 sf

char
single character, single quote 'x'
store any char from Unicode

Unicode
16 bits/character, 2^{16}

ASCII
upper/lowercase, punctuation, num, symbols, control characters

boolean
Java 里 T/F 都是小写 — true, false

expressions
只要其中有一个浮点数 → 答案浮点数

y = x++ postfix increment y=x, x+=1

y = ++x prefix increment x+=1, y=x

Data Conversion

1) Widening = safe, small → big.

2) Narrowing = big → small

- Assignment Conversion: widening only, don't change val

```
int A=20;
double B=A;
```

- Promotion Conversion: during expressions

```
int A=10;
float B=1.2;
rst = A/B;
```

- Casting Conversion: dangerous! widen/shorten

```
int A=10;
float B= (float) A/6;
```

new
create an object from a class

e.g. Scanner Scan = new Scanner (System.in);

then can be used w methods

e.g. ans = Scan.nextLine();

Using Classes & Objects

eg. String title;
→ create object reference variable, hold address.
instantiation = create new object
object → instance of a class

[primitive var → contain val [when copy, copy val]
object var → contain address of obj. (obj ref → pointer). [copy address]
when ≥2 ref same obj → aliases

garbage collection: useless obj., Java automatically return res memory
special syntax for string: title = "don't need to use new";
string obj. immutable = 不能改值/常值
* .substring (start); // 从 num index 开始.
.substring (start, stop-before);

* Random gen = new Random();
num = gen.nextInt(10); // 0-9

Java class library / Java API (Application Programming Interface)
classes → packages (java.lang auto-import)

enumerated types
eg. enum Season {winter, spring, summer, fall}; // enum: type-safe, can't assign num
Season time;
time = Season.fall; // fall: ordinal value 3.

wrapper classes

primitive	wrapper
byte	Byte
short	Short
int	Integer // 全称!
long	Long
float	Float
double	Double
char	Character // 全称!
boolean	Boolean

* Autoboxing: convert primitive to wrapper
e.g. Integer obj; e.g. Double val = 15.75;
int num = 42;
obj = num;

e.g. Integer age = new Integer(40); // object can serve as containers of other objects
num = Integer.parseInt(str); // static methods that help manage type.

Writing Classes.

class contain data declaration & method declaration

Constructors

- set up an object when first created e.g. set initial value to variables, etc.
- has the same name as the class, no return type !! (default constructor accept no parameters)

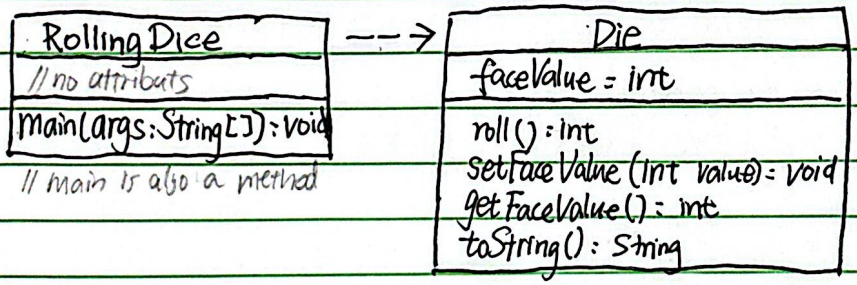
data scope

- area in the program in which that data can be referenced.
- data declared at class level: can be referenced by all methods in that class (instance data)
  data declared within a method: only within method, (local data).
- each obj has own faceValue variable → has its own state

UML Unified Modeling Language.

UML diagrams — relationship classes ⇔ object.

- consists of: ≥ classes (each has class name, attributes/data, operations/methods).
- lines between classes represent associations
  dotted arrow shows one class uses the other (call its methods)

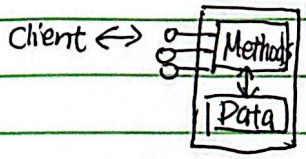


Encapsulation

2 views of obj

- internal: var, methods
- external: services the obj provides, how it interacts. ⇒ obj is encapsulated, providing services
  ⇒ services define object interface

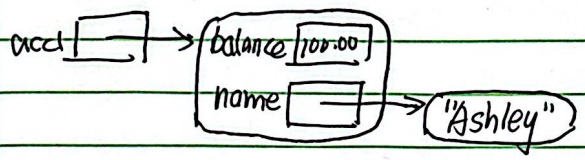
- client should be able to access obj var directly
  changes to obj state (variables) only made by its methods
  ⇒ obj is self-governing, like a black box



- How to accomplish encapsulation
  ⇒ use of **visibility modifiers**
  - public — referenced anywhere, service methods
  - protected — involve inheritance, support method
  - private. — referenced within that class/etc., support method
  - default visibility — referenced by any class in the same package.

* instance var shouldn't be declared w public visibility.
  - accessor — return current val of var.
  - mutator — changes val of var. (might have limit to extent of change

Method declarations
- return type method name (parameter list parameter type+name)
- name of parameter in method declaration → formal parameter → automatic local var.



class example:

```
public class Die{
    private int faceValue;
    public Die(){
        faceValue = 1;        ⇒ Constructor
    }
    public int roll(){
        faceValue = (int) Math.random()*6+1);    ⇒ methods
        return faceValue;
    }
}
```

Features of applications

User interface (GUI), Icons, Pointer (鼠标指针), Window (document, program, message), Menus (actions user can perform), Menu Bar (help navigate), Pull-down/drop-down menu, Dialog Box (input, selecting, etc), Help, Toolbars, Buttons (interactive, initiate actions), Standard Toolbar

Conditionals & Loops

- Relational operators: ==, !=, <, >, <=, >= logical operator: !, &&, ||
- if (condition){ while( )
  ——; ——;
  }

  short-circuited operators
  if left operand sufficient to determine result, right operand is not evaluated

```
if (  )
  ——;
else if(  )
  ——;
else
  ——;
```

- Random generator = new Scanner (System.in);
  int ans = generator.nextInt(int Max); [0, bound) 整数
  double ans = generator.nextDouble(); [0.0, 1.0)
- Comparing strings
  if (name1.equals(name2)) #不能用 ==
  ——; # lexicographic order
  name1.compareTo(name2) # return 0 → 相等 string, return neg. val. name1 < name2, 反之亦然
- Comparing objects
  用 == return true if the 2 references are aliases of each other
  用 equals 也是一样的
- overriding functions
  可以重新定义 eg. toString(), equals() to public String toString() { }
- string operations
  str1.length()
  str.charAt(n)

- **iterator**

allow collect items one at a time

e.g. Scanner
.hasNext() // type是boolean.
.next() // type是String

- **ArrayList class**

part of java.util.

can grow/shrink as needed

methods:

- add (int ind, E obj) e.g. str.add("Pete") str.add(2, "Bingo")
- remove (int index) eg. int loc = str.indexOf("Pete"); band.remove(loc);
- get (int index) eg. Str.get(index)
- isEmpty()
- (int) size() eg. str.size()

ArrayList<String> names = new ArrayList<String>();

* ArrayLists don't store primitive types! Use wrapper class

- **switch**

```
switch (expression)
{
   case value1:
        statement;
   Case value2:
        ——;
}
```

每个case最后要加break
不然会继续下个case
还可以有个 default: {—}.
所有的不通过前述原个

- conditional operator

condition ? expression1 : expression2
T → expression1
F → expression2

- do

{—} ⇒ execute 为1次
while (condition)

- **for**

for (initialization; condition; increment)
——;

[for-each loop example] ArrayList.
for (Book myBook : bookList)
——;

Object-Oriented Design

Static method — invoked through class name.

e.g. methods of Math class → static. result = Math.sqrt(2)

declare static method & variable → associate it w the class rather than with an object of that class

static — 全class共享的变量/method，不同的obj还是取得同一个 e.g. ClassA.y = 8890;

instance — 某具体实例 e.g. ClassA obj1 = new ClassA(); obj1.y = 8890;

usually obj has its own data space

Static → only 1 copy of var. exists → space created when class is first referenced

→ all obj instantiated from class share static variables, change val in 1 obj. changes it for all others

- Static method can't reference instance variables ∵ instance var don't exist until an object exists, don't need obj to execute a static method.
- Static method can reference static variables / local variables

---

Class relations

Dependency: A uses B (involve methods of another class)

Aggregation: A has a B

Inheritance: A is a B

△ **Dependency**

can exist between obj of the same class

method of a class can take obj of same class as a parameter.

e.g. concat method of String class takes another String obj as parameter.

```
Str3 = str1.concat(str2);
r3 = r1.reciprocal();
```

在同个class的method可以用另一个method.

Aggregation

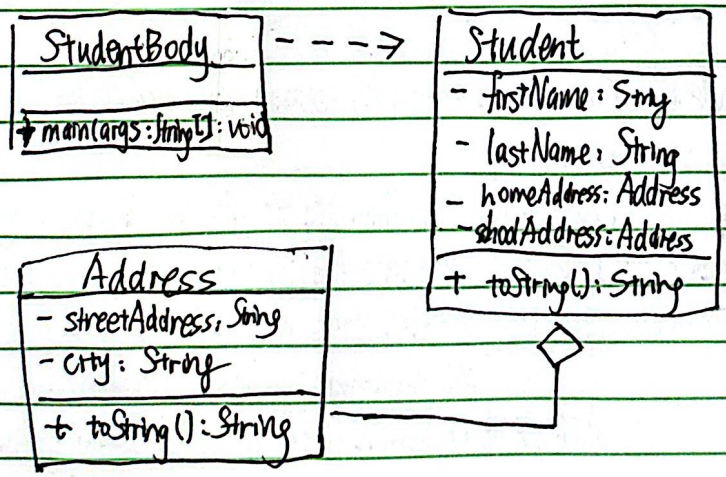
obj made up of other objects.
reference other objects as instance data
*special kind of dependency (rely on the objects that compose it)

e.g. 在 Student class 里面

```
public class Student
{
    private String firstName, lastName;
    private Address homeAddress, schoolAddress;
    public Student (String first, String last, Address home, Address shool)
    {
        firstName = first;
        ____;
        ____;
        ____;
    }
    这里还有个 toString()
```

然后是 Address class

```
public class Address ____ ____ ____
{
    private String streetAddress, city;
    public Address (String street, String cityAdd)
    { ____;
      ____;
    }
    public String toString()
    {
        ____        // String 可以 + = 的
    }
```



△ this → reference to object itself.

在method里用，则指回用 method 的 object

distinguish instance variables of a class from corresponding method parameters w same names

e.g.
```
public Account (String name, double balance, long acctNumber)
{
    this.name = name;
    this.acctNumber = acctNumber;
    this.balance = balance;
}
```

△ Interface

- Collection of abstract methods & constants.; establish set of methods a class will implement
- abstract method: method header without a method body

虽然能用 abstract 这个 modifier declare，但因为 interface 都是 abstract methods，所以可省略

e.g.
```
public interface Doable
{
    public void doThis();      //记得 method header 后在这儿有 semicolon
    public int doThis2(double value, char ch);
}
```

- interface cannot be instantiated (can't create obj from it); no concrete implementation

methods in an interface have public visibility by default

class implements by providing implementations for every abstract method in interface

e.g.
```
public class CanDo implements Doable {
    public void doThis()
    {}
    public void doThat()
    {}
}
```

- can also contain constants (when implemented, class gain access to all its constants)

class can implement multiple interfaces

- Java API interfaces: eg. Comparable interface 有个 abstract method = compareTo 可以 implement

method perform iteration ← Iterator interface 有三个 method: hasNext, next, remove (remove 最近被 next return 的值)

object provide iterator ← Iterable establish that an object provides an iterator, loops.

有个 method iterator, return an Iterator object; implement for each

*你不一定要 implement interface 才 implement certain methods (eg. compareTo)

△ Enumerated Types

e.g1 enum Season {winter, spring, summer, fall} // 后面无semicolon!

eg2
```
class RandomStuff{
    enum Season {winter, autumn}   // outside method.
    public static void main(String[] args){
        Season a;
        a = Season.winter;   //不能new一个enum出来
        System.out.println(a);
    }
}
```

eg3. like classes, can add additional instance data & methods, can also define enum constructor

```
public enum Season{
    winter("Dec. to Feb.");  →这儿是逗号哦
    autumn("Sept. to Nov.");
    private String span;
    // constructor
    Season (String months){
        span = months;
    }
    public String getSpan(){
        return span;
    }
}

public class SeasonTester{
    public static void main(String[] args){
        for (Season time : Season.values())
            System.out.println(time + "\t" + time.getSpan());
    }
}
```

- enumerated type all have static method called values() → return all possible values of type the list return can then be processed in for-each loop.

△ copy of actual parameter (value) is stored into formal parameter (in the method header)
object passed to a method → actual & formal parameter become aliases
e.g. 你把 obj.setValue(888); 那初始值也被改了 .setValue 这里是自己定义的，obj是下载义class Numbers

△ **method overloading.**
Give a method name multiple definitions.
Signature → include number, type & order of the parameter. (don't include return type!)
Signature of each overloaded method must be unique.

e.g.
```
float tryMe (int x)
{
  ___;
}

float tryMe (int x, float y)
{
  ___;
}
```

- constructors can be overloaded

o Defect testing : execution of test cases to uncover errors
Regression testing : after fixing errors, running previous test suites (make sure no new errors)
Black-box testing : test cases developed w/out considering internal logic
input can be organized into equivalence categories.
(2 input in same equiv. category → similar result)
∴ good test suite cover all equiv categories & focus on boundaries between categories
white-box testing : focus on internal structure of the code.
goal → every path thru code tested.
(paths are governed by any conditional/looping statements
good testing effort → black-box & white-box tests

Arrays

△ Declaring & Using Arrays
ArrayList → class in Java API

2D-array
int[][] grades = {{-}, {-}};

object - attribute & behavior, instantiation of class, have memory space
class - multiple obj can derive from same class, blueprint

```
public String[] getComplaints(){
    return this.complaints;
}
```

- dynamic binding / late binding
- Defers method binding until run time

```
int[] scores = new int[6];
String[] words = new String[3]
```

main → String as parameter
(int ... list) as parameter)

Polymorphic reference
→ var refer to diff type of objects at diff pts in time
→ reference type e.g. Holiday
object type e.g. Christmas
- call function 因為是 object type.
→ Compiler 看的是 ref type.

```
public abstract class Product
{                ↳ can't be final/static.
    // class contents
}
```

Can use interface to set up polymorphism